2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA) | 979-8-3503-2658-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/ISCA59077.2024.00024

FEATHER: A Reconfigurable Accelerator with Data Reordering Support for Low-Cost On-Chip Dataflow Switching

Jianming Tong
Georgia Institute of Technology
Atlanta, Georgia, USA
jianming.tong@gatech.edu

Anirudh Itagi
Georgia Institute of Technology
Atlanta, Georgia, USA
aitagi7@gatech.edu

Prasanth Chatarasi
IBM Research
Yorktown Heights, USA
prasanth@ibm.com

Tushar Krishna
Georgia Institute of Technology
Atlanta, Georgia, USA
tushar@ece.gatech.edu

***Abstract*—The inference of ML models composed of diverse structures, types, and sizes boils down to the execution of different dataflows (i.e. different tiling, ordering, parallelism, and shapes). Using the optimal dataflow for every layer of workload can reduce latency by up to two orders of magnitude over a suboptimal dataflow. Unfortunately, reconfiguring hardware for different dataflows involves on-chip data layout reordering and datapath reconfigurations, leading to non-trivial overhead that hinders ML accelerators from exploiting different dataflows, resulting in suboptimal performance. To address this challenge, we propose *FEATHER*, an innovative accelerator that leverages a novel spatial array termed *NEST* and a novel multi-stage reduction network called *BIRRD* for performing flexible data reduction with layout reordering under the hood, enabling seamless switching between optimal dataflows with negligible latency and resources overhead. For systematically evaluating the performance interaction between dataflows and layouts, we enhance Timeloop, a state-of-the-art dataflow cost modeling and search framework, with layout assessment capabilities, and term it as Layoutloop. We model *FEATHER* into Layoutloop and also deploy *FEATHER* end-to-end on the edge ZCU104 FPGA. *FEATHER* delivers $1.27 \sim 2.89\times$ inference latency speedup and $1.3 \sim 6.43\times$ energy efficiency improvement compared to various SoTAs like NVDLA, SIGMA and Eyeriss under ResNet-50 and MobiletNet-V3 in Layoutloop. On practical FPGA devices, *FEATHER* achieves $2.65/3.91\times$ higher throughput than Xilinx DPU/Gemmini. Remarkably, such performance and energy efficiency enhancements come at only 6% area over a fixed-dataflow Eyeriss-like accelerator. Our code is released at https://github.com/maeri-project/FEATHER.**

***Index Terms*—Reconfigurable Accelerator, Dataflow, Layout**

I. Introduction

The field of Machine Learning (ML), specifically Deep Neural Networks (DNNs) is pervasive today across image classification [12], [40], object detection [3], [37], text summarization [20] and sentiment analysis [25]. Such a plethora of ML models introduces great diversity in structure (serial or parallel layers connectivity), layer types (depth-width, point-width, dilation convolutions, or even a fusion of them), and sizes (number of channels, kernels, height, and width) [7], [49].

The mechanism for orchestrating a DNN layer over the accelerator's on-chip compute and memory resources is called *dataflow*. It can be precisely defined by transformations of the loop nest, as shown in Fig. 1. Several prior works [33], [41] have demonstrated that dataflows can lead to significant differences in compute utilization and up to two orders of magnitude variance in latency and energy, and thereby motivated the need to support per-layer dataflow flexibility.

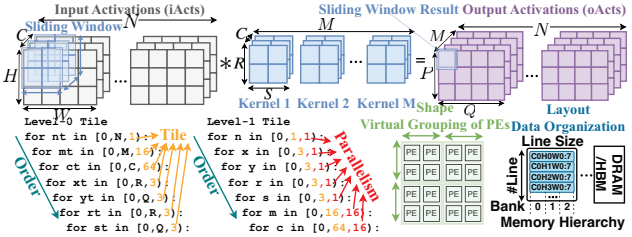


Fig. 1: Terminology of convolution workload and dataflow

Changing dataflows on accelerators requires (a) reconfiguring datapaths in computation, distribution, and reduction networks, and (b) modifying data layout in on-chip buffers. Almost all prior works have focused on the first aspect, and several clever interconnect topologies for data distribution and reduction have been proposed that activate subset of paths at runtime through reconfiguration depending on the dataflow being run [42], [44]. However, data layout in the on-chip buffer is a critical and often overlooked in past work.

In this work, we demonstrate that the high performance of dataflows is unachievable in practice without layout reordering capability. This is because, without a suitable data layout, the required data may be located in the same SRAM banks and compete at the same SRAM reading ports. Such bank conflict slows down the delivery of data to computation engines, leading to stalling and computation underutilization. Overlooking layout reordering thus introduces a significant $128\times$ performance gap between theory and practice as quantified in Fig. 2. We discuss this with more depth in §II.

Unfortunately, layout reordering comes with severe latency and energy overheads. Off-chip layout reordering requires back-and-forth data movement between off-chip DRAM/HBM and computation, while on-chip layout reordering requires additional intermediate storage and extra latency in the critical path. In fact, these costs can outweigh the benefits of switching dataflows, leading existing ML accelerators to compromise settling on a single dataflow (e.g., Xilinx DPU, Gemmini, NVDLA, Eyeriss in Table I) that provides good average utilization across all layers, but sub-optimal performance.

To unleash optimal performance, we propose a novel accelerator *FEATHER*, Flexible Engine for Acceleration of Tensors with Hardware Element for Reordering, which includes a novel reconfigurable reduction network called Butterfly Interconnect

979-8-3503-2658-1/24/$31.00 ©2024 IEEE
DOI 10.1109/ISCA59077.2024.00024

TABLE I: Feature comparison: how FEATHER resolves challenges of prior works without on-chip layout reordering.

Work	Dataflow Switching	Layout Reorder	Challenge	***FEATHER*** solution (key component)
NVDLA [39]	✗	no reorder	underutilization from fixed parallelism	flexible dataflows (*NEST*)
Xilinx DPU [51], Gemmini [21]	✗	no reorder	linear reduction	parallel logarithmic reduction (*BIRRD*)
SIMBA [47], Eyeriss [13]	✗	no reorder	load imbalance across PE	pick load-balance dataflows (*NEST*)
Eyeriss_v2 [15], SARA [44]	✓	off-chip	high latency of moving data off-chip	on-chip reordering with latency hidden (*BIRRD*)
MAERI [35], SIGMA [42]	✓	off-chip	long wires of reduction network	small standalone reduction network (*BIRRD*)

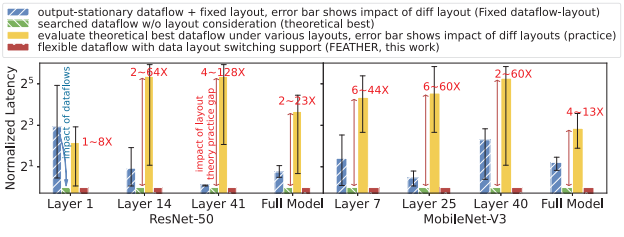


Fig. 2: Latency evaluation of dataflows on 16×16 PE array with various layouts (error bar shows layout impacts, less latency is better). The best flexible dataflow (green bar) *theoretically* reduces overall latency of fixed dataflow-layout (blue bar) by 63.3%. However, ignoring the impact of layout considerations in theoretical dataflows results in up to a $128\times$ latency gap in *practice* (yellow bar). FEATHER eliminates the gap by co-switching dataflow-layout (red bar).

for Reduction and Reordering in Dataflows (*BIRRD*). With *BIRRD*, the latency of layout reordering is completely hidden in data reduction, allowing data layout in on-chip storage to be manipulated for the demand of optimal dataflow without any latency costs. We call this approach as reordering in data reduction (RIR). Thus, *FEATHER* fully achieves the theoretical performance of optimal dataflows without incurring bank conflicts. Furthermore, *FEATHER* also pioneers a new paradigm to co-switch both dataflows and data layouts at layer granularity, with minimal switching overheads. This ability to accommodate low-cost layout-dataflow co-switching is, as far as we know, unsupported by any existing accelerator.

To fully explore the potential of *FEATHER*, we also developed a tool that facilitates: (a) dataflow evaluation factoring in data layout, and (b) (layout, dataflow) co-exploration.

Our key contributions can be summarized as follows:

- We demonstrate the interaction between dataflows and data layouts, motivating the need for data reordering support within reconfigurable dataflow accelerators. We further categorize existing reordering patterns and implementations (§II).
- We present a novel accelerator *FEATHER* with several novel features (§III). First, a neural engine with temporal local reduction and spatial forwarding, *NEST*, for dataflow flexibility. Second, a multi-stage network called *BIRRD* enabling flexible reductions from arbitrary groups of multiple inputs to multiple results, at lower area overhead compared to prior works with similar capabilities. Further, BIRRD supports Arbitrary reorder via a novel technique RIR, that completely conceals data layout reordering latency behind reduction (§IV).
- We extend a state-of-the-art accelerator modeling framework Timeloop [41] with support for physical on-chip storage, layout representation, and dataflow-layout co-search. We call this new framework *Layoutloop* (§V) and use it for our evaluations.
- We implement and deploy *FEATHER*, end-to-end, on an edge ZCU 104 FPGA device and also model it using *Layoutloop*. *FEATHER* achieves $1.27\sim2.89\times$ inference latency speedup and $1.3\sim6.43\times$ energy efficiency improvement compared to various SoTAs across multiple DNN models, and $2.65\times/3.91\times$ more throughput than Xilinx DPU/Gemmini on real FPGAs. On average, efficient pairs of (dataflow, layout) results in an energy savings of 27% to 33% across workloads despite the energy costs of layout reordering. Remarkably, all enhancements come at only 6% area over a fixed-dataflow Eyeriss-like accelerator.

TABLE II: On-chip memory terminology

Term	Meaning
Buffer	A logical 2D on-chip memory (num_line × line_size) stacking multiple SRAM banks both vertically (num_line) and horizontally (line_size).
Bank	A physical 2D SRAM (entries × io) with address/data ports.
Line/Row	A buffer line (line_size = accumulated IO of horizontal SRAM banks).
Port	An input/output port, each bank has at most two ports in TSMC 28nm.

II. Background and Motivation

A. Dataflow Space in Convolution

Fig. 1 depicts a convolution operation with seven dimensions with various shapes. Dataflows can be represented as a nested loop with four types of optimizations [24], [34].

- **(T)iling** breaks down dimensions of iActs N,C,H,W into smaller chunks, and enables executing workloads in tile granularity as on-chip storage is limited.
- **(O)rdering** allows arbitrary loop reordering (aka "stationarity" [13]) to reuse more data since dimensions N,M,C,P,Q,R,S do not come with loop-carried dependencies except reduction-dependencies over C, R, and S.
- **(P)arallelism** allows for arbitrary parallelism over any dimensions as all dependencies are loop-independent, leading to different spatial reuse opportunities.
- **(S)hape** defines the virtual grouping of the physical PE array.

These *dataflow flexibility (TOPS)* [34] create an extremely large dataflow design space with a complexity of $O(10^{36})$ for a single convolution layer [27]. The choice of the dataflow affects both runtime performance (as it affects overall compute utilization) and energy efficiency (as it affects the number of accesses across the memory hierarchy). Not surprisingly, no single dataflow is generally optimal for all types of layers given their diverse sizes and shapes [33], [41]. This can be seen by comparing the first two bars (blue and green bars) in Fig. 2.

B. Data Layout in on-chip Storage

Various organizations of on-chip storage are logically a 2D buffer (Tab. II), where the width of each logical buffer row, termed "line size", represents bandwidth (max number of data

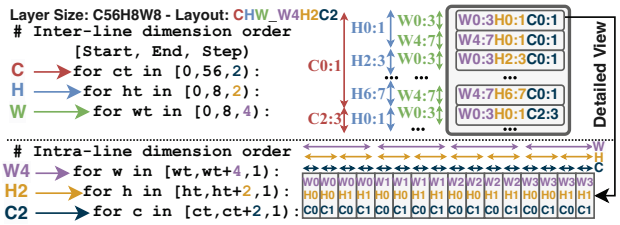


Fig. 3: Layout terminology example: 'CHW_W4H2C2'. 'CHW' signifies the inter-line dimension order as C→H→W across lines. 'W4H2C2' indicates the intra-line dimension order: (4,2,2) elements from the (W,H,C) dimensions are flattened into a single row in the order of W→H→C.

words a buffer could supply per cycle) and the depth represents the total number of buffer row entries as shown in Fig. 1.

Physically, on-chip storage is implemented by BRAM/URAM in FPGA and SRAM in ASICs, which come with a *fixed number (often two) read or write ports*. Therefore, once arranged into the logical 2D buffer, the number of lines being concurrently accessed is limited by the number of ports. A request that accesses more lines than the available ports will lead to bank conflicts, resulting in a slowdown from the reading/writing delay (resource hazard).

Data Layout Terminology. In this paper, data layout is represented as "(Inter-line dimension order)_(Intra-line dimension order interleaved with sizes)" with one example shown in Fig. 3. For instance, two commonly used PyTorch data layouts, channel-last [18] and row-major [38], can be interpreted as Channel (C) or Width (W) being the innermost dimension in both inter and intra-line orders, separately.

C. Interaction of Dataflow and Data Layout

In the rest of the paper, we refer to a (dataflow, layout) pair with bank conflicts as ***discordant***, whereas its non-conflicting counterpart is termed ***concordant***, i.e. a layout is concordant to a dataflow if there are no bank conflicts. And we use ***concordant dataflow space of a layout*** to refer to all concordant dataflows choices under a layout. Switching optimal dataflows for different layers is not trivial given that it necessitates a costly reordering to convert the data layout into a concordant form to prevent bank conflicts.

In this subsection, we discuss some crucial insights, underscoring the necessity of co-switching dataflows-layouts for different layers by evaluating the performance of various combinations of dataflows and data layouts as shown in Fig. 4.

Insight 1: Discordance between dataflow and data layout leads to bank conflicts and results in performance degradation.

A discordance between dataflow and data layout leads to slowdown because compute units have to stall and wait for data to arrive, as illustrated by the slowdown from green bar to yellow bar in Fig. 2. Taking ResNet-50 layer 47 as an example (Fig. 4-M7), the channel-parallel dataflow requires concurrent access to iActs (H0W0C0:3), which are distributed across four separate lines, including line 0, r4, r5 and r6, in the row-major layout (Fig. 4-L4). Therefore, a 0.5 slowdown is encountered, resulting in 50% practical computation utilization. Such bank conflicts cannot be resolved by line rotation, since moving one conflicted line to another bank leaves the remaining three lines still in conflict. This slowdown analysis also applies to Fig. 4-M1,2,3,6.

Insight 2: Co-switching (dataflow, layout) for different layers is necessary for high performance with optimal efficiency.

For certain workloads, picking a fixed layout might not suffer a slowdown from bank conflicts, like choosing row-major layout for both two layers of ResNet-50 (M4 and M8 in Fig. 4). However, the mapping M5 ("*FEATHER*'s pick") delivers better energy efficiency than M8 as it supplies data with reading less number of lines. Therefore, even under a small parallelism of four, co-switching dataflows and layouts is essential to maximize performance and energy efficiency. Practical designs (e.g. 128×128 systolic array in Google TPU) will further amplify such a need as it brings higher parallelism in more dimensions and requires more concurrent data.

Insight 3: Systematic layout modeling should be factored into dataflow exploration for bridging the theory-practice gap.

Dataflow has a huge space, which requires systematic modeling and searching algorithms to identify the optimum. However, many dataflow exploration frameworks [33], [41] and algorithms [9], [27], [28], [30] purely model on-chip storage as bandwidth, often assuming ideal data layouts, which could lead to significant theory-practice performance gap. For instance, all layouts in Fig. 4 possess identical bandwidth, but they result in markedly different compute utilization and energy efficiency for two workloads, which is not the case in the existing frameworks as they do not model layout. In Fig. 2, we find that the best dataflow reported by a mapper from an existing framework [41] (green bar), can in practice perform 2 orders of magnitude worse (yellow bar) than the fixed dataflow case (blue bar) due to the discordant accesses to the on-chip memory. Thus, taking layout into consideration *during* search (red bar) is necessary and crucial.

D. Data Reordering Patterns

1) Reorder Target (iActs): As established above, both weights and input activations (iActs) necessitate layout reordering within the on-chip memory when switching dataflows. For ML inference, the structure and weights of ML models are established prior to deployment, enabling the offline optimal dataflow-layout determination for each layer and offline reordering of all weights. Consequently, an optimal layout for weights within the on-chip scratchpad is assured. However, iActs are generated in real-time, so that iActs reordering happens online. Therefore, this work focuses on layout reordering of iActs.

2) Reorder Patterns vs. Implementations: Layout transformations require certain reorder capabilities, referred to as reorder patterns. A reorder pattern has different hardware implementations *with different critical-path latency*. To decouple the concept of reorder patterns from their physical implementations, we analyze reordering in two steps: (1) categorize reordering

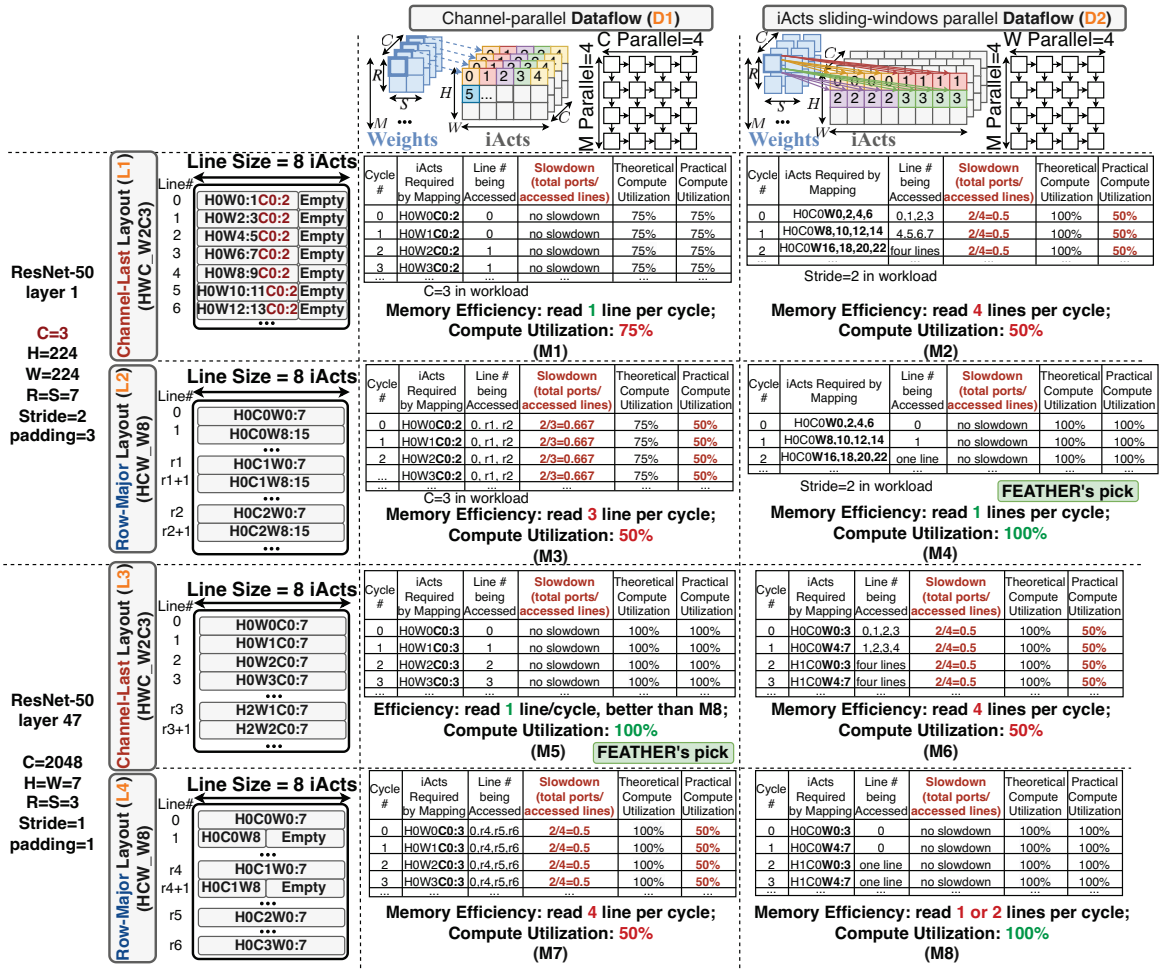
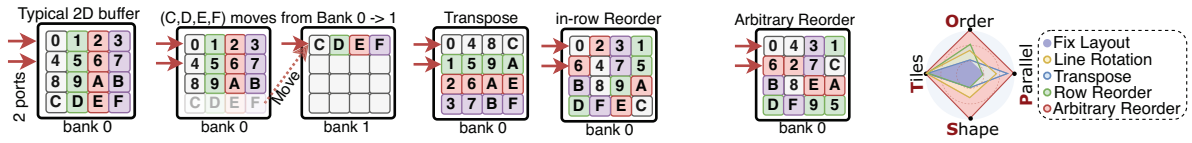


Fig. 4: Memory efficiency and computation utilization of various **(workload, dataflow, data layout)** combinations on weight-stationary 4×4 Systolic Array (SA). **Dataflows:** input channel-parallel (D1) and sliding-window parallel (D2). Dataflow D1/D2 reads at most four iActs from C/W dimension concurrently from the on-chip buffer every cycle, separately. The digit in iActs indicates the cycle index such iActs get read. **Workloads:** (1) ResNet-50 layer 1 with a large height and width, and (2) ResNet-50 layer 47 with a large channel number. **Layouts:** channel last-layout (L1, L3) and row-major layout (L2, L4). In the channel-last layout, data from different input channels (dimension C) are spread across an individual line, while in the row-major layout, multiple data from different input width (dimension W) are flattened. The performance of mappings (M1$\sim$M8) for different (workload, dataflow, layout) combinations are analyzed in the tables. In each table, "iActs Required by Mapping" lists all iActs that need to be concurrently read from on-chip buffer every cycle, and the corresponding index (#) of lines being accessed are listed in "Line # being Accessed". We assume dual read ports (because TSMC offers SRAM with at most two ports), such that a concurrent read for more than two lines leads to slowdown, which reduces "Theoretical Computation Utilization" (estimated as mapping efficiency over the array) into "Practical Compute Utilization" (computed as multiplication of theoretical utilization with slow down). **Takeaway:** For optimal performance, co-switching (dataflow, layout) is crucial, because *dataflow* matters (comparing M1 vs. M4), and *layout* also matters (comparing M2 vs. M4).

into distinct functional patterns, as illustrated in Fig. 5, and analyze its impact on dataflow flexibility in §II-D3. (2) pinpoint specific hardware implementations to these patterns in §II-E.

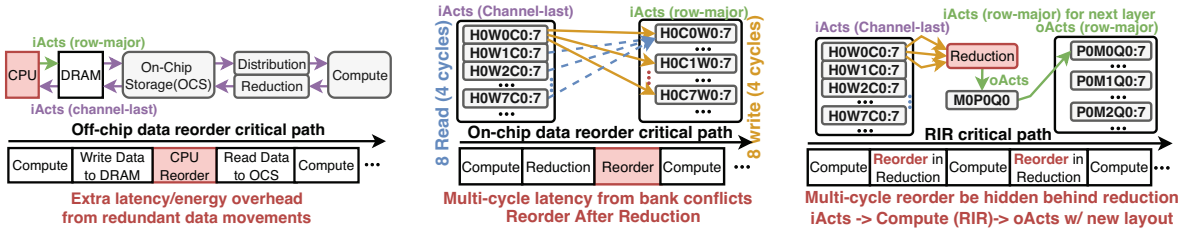
3) Impact of Reorder Patterns on Dataflow Flexibility: A fixed layout has limited concordant dataflow space, restricting fully-flexible accelerators to less-performant dataflow choices. To improve performance, reordering is required to enlarge concordant dataflow space with more flexibility in TOPS.

- **Fixed layout** (Fig. 5a) is only concordant to dataflows which concurrently access up-to two rows within a single bank, such as $(0, 1, 2, \cdots, 7)$. This restricts concordant dataflow space to limited T,O,P,S flexibility (see purple quadrilateral in Fig. 5f).
- **Line Rotation** (Fig. 5b) arguments concordant dataflow space



(a) Initial layout (b) Line Rotation (c) Transpose (d) Row-Reorder (e) Arbitrary Reorder (f) Concordant Dataflow Space

Fig. 5: Overview of reordering *patterns*. The 2D layout without any reordering is shown in 5a, which only allows reading two rows concurrently, assuming true dual-port SRAM. Line Rotation (5b, e.g., Medusa [48]) moves a row from bank 0 to bank 1 prior to reading, enabling simultaneous access to at most three rows from bank 0 through dual-bank ports. This technique, however, utilizes additional port from bank 1, potentially limiting access to other data in bank 1. Transpose (5c, e.g., MTIA [19] and TPUv4i [26]) could swap rows with columns. Row Reorder (5d, e.g., TPUv4i [26]) permutes data within each row. Arbitrary reorder (5e, proposed in this work) enables arbitrary permutation for data within the entire 2D buffer. Line Rotation, Transpose and Row-Reorder are done by prior works by reading at most two rows per bank, leverage Transpose/Permute unit to reorder and then write data back in concordant order (On-chip RAR in 6b). In contrast, *FEATHER*'s *BIRRD* network (§III-B) performs the Arbitrary-Reorder during the reduction phase of the matrix multiplication or convolution computation (RIR in Fig. 6c). The concordant dataflow space supported by each layout reorder pattern is shown in 5f. *Reordering enables a given layout to alter the order of data it could provide* ***per cycle*** *and* ***across cycles***. Among four dimensions (T,O,P,S) of concordant dataflow space, reordering enlarges O,P,S by supporting dataflows to read from or write to layout in different order. Note that reordering by itself cannot enlarge T dimension flexibility because higher Tiles flexibility requires accessing more data per cycle.



(a) Off-chip Data Reorder. (b) Reorder after Reduction (prior works). (c) Reorder in Reduction (RIR, this work).

Fig. 6: Comparison of data reordering *implementations*. This work proposes RIR that eliminates reorder latency and bank conflicts. We discuss on-chip reorder patterns, including transpose, line rotation, row-reorder and arbitrary reorder, in Fig. 5.

to concurrently access up-to **three** rows within a single bank by storing a copy of a row in other banks. For example, to access three rows including data $(0,1,\cdots,7,C,D,E,F)$ from bank 0 in Fig. 5b, row (C,D,E,F) is moved to bank such that it provides $(0,1,\cdots,7)$ from bank 0 and (C,D,E,F) from bank 1 to avoid bank conflicts. However, line rotation comes at the price of (1) extra bandwidth: it employs three ports for reading data that could be accessed with up-to two ports under concordant layout, (2) storage: it stores a copy of (C,D,E,F). Such price could have been used for supporting more parallelism under arbitrary reordering to improve performance.

- **Transpose** (Fig. 5c) enables concurrently access to up-to two rows *or columns* within a bank, hence augmenting concordant dataflow choice with higher P flexibility than fixed layout. But pure transpose falls short of supporting tiled layout transformation, such as changing layout from HWC_W2C3 (Fig. 4, L1) to HWC_W8 (Fig. 4, L2)
- **Row Reorder** (Fig. 5d) does not support more concurrent access within a single bank, but enables arbitrary order within each row, hence supporting dataflows with higher O flexibility. Further, row reorder also supports im2col [11], which does not reduce bank conflicts because it still accesses the same number of rows from on-chip buffers.
- **Arbitrary Reorder** (Fig. 5e) enables arbitrary layout transformations, hence making all dataflows concordant with full-fledged O,P,S flexibility, as shown by red diamond in Fig. 5f.

TABLE III: SoTA on-chip reordering vs. *FEATHER*.

Work	Dataflow	On-chip Reorder Patterns	Implement
im2col [11]	N/A	Row-Reorder (Fig. 5b)	RAR
Medusa [48]	N/A	Line Rotation (Fig. 5b)	RAR
MTIA [19]	TOP	Transpose (Fig. 5c)	RAR
TPUv4 [26]	TO	Trans.+Row-Reorder (Fig. 5d)	RAR
This Work	TOPS	Arbitrary Reorder (Fig. 5e)	**RIR**

E. Data Reordering Implementations

The layout reorder patterns described in Fig. 5 could have different implementations with *different critical-path latency*.

1) ***Existing Implementations****:* We classify existing reordering implementations into three categories.

a) No Reordering: If there is no reordering, either the accelerator needs to run a fixed dataflow or a subset of dataflows that are concordant to the fixed layout, or pay the cost of bank conflicts due to discordant accesses. This can lead to sub-optimal performance (as shown by blue bar in Fig. 2).

b) Off-chip Reordering: SoTA that support dataflow switching (Tab. I) require iActs to move to off-chip DRAM, get reordered there by CPU, and then move back to the accelerator. This naturally incurs extra latency and energy costs (Fig. 6a).

*c) On-chip Reorder After Reduction (**RAR**):* Existing on-chip reordering techniques essentially perform reordering after reduction. The post-reduction oActs are first written to the on-chip buffer, then read and sent to a separate unit to perform a layout transformation, and then fed back to compute unit as iActs of the next layer. This puts reordering in the critical path, as shown in Fig. 6b. Previous arts all fall into this bucket with *explicit reordering latency*, as listed in Tab. III. For example, Medusa [48] proposes dedicated hardware between on-chip buffer to compute unit to implement line rotation (Fig. 5b); Meta's MTIA [19] proposes a Memory Layout Unit (MLU) to implement transpose; Google's TPUv4 [26] also supports row-reordering (Fig. 5d) to facilitate im2col.

*2) **Proposed Implementation** - On-Chip Reorder In Reduction (RIR):* This work proposes to perform reordering on output during reduction phase of computation, such that oActs are written in the layout concordant with the dataflow of the next layer. We call this Reorder in Reduction (**RIR**). RIR *implicitly* modifies the layout during the reduction process when ***generating oActs*** instead of transforming iActs from one layout to another, as depicted inFig. 6c. This approach (i) removes reordering from critical path, (ii) reduces the total number of partial sums into fewer final sums, reducing buffer access and effectively minimizing potential bank conflicts. §IV provides more details.

F. Inefficiency of SoTA Reconfigurable Dataflow Accelerators

Data Reordering Support. Driven by the observation that on-chip dataflow plays a crucial role (§II-A), there has been a suite of past work on accelerators with hardware support for running diverse dataflows [32]. Their key observation is that different dataflows trade-off spatial and temporal reuse, and thereby flexible dataflow requires support for different operand stationarity within buffers and variable-sized spatial and temporal reductions through the interconnect. Unfortunately, these accelerators have **two** limitations as elaborated in §II-D and §II-E: (i) either they do not support any on-chip reordering (Tab. I) or support limited transformations including transpose, line rotation or row-reorder (Tab. III). This work extends support to arbitrary reordering. (ii) prior on-chip reordering support can cause bank conflicts, increasing reordering time. This work removes reordering from critical path by doing it during the reduction phase of the computation.

Dataflow-Layout Co-Search. There has also been a suite of dataflow/mapping search tools [23], [27], [41] that can recommend the optimal dataflow given a layer and hardware resources. *However, none of these tools explore on-chip data layouts as part of the search process.*

Contributions of this work. This work addresses the aforementioned gaps via three key contributions: (i) a reconfigurable accelerator *FEATHER* with a novel on-chip fabric called *BIRRD* that provides support for *both* dataflow flexibility and layout flexibility through arbitrary reorder, (ii) a new on-chip data reordering mechanism called RIR (implemented by *BIRRD*) whose key goal is to *generate* data in the layout required by the next layer instead of explicitly requiring layout conversion

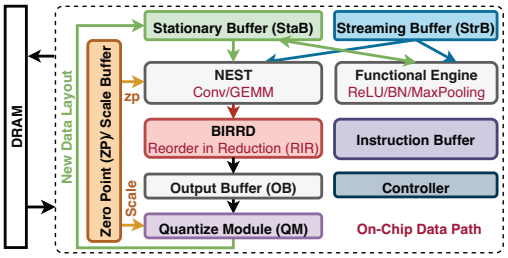


Fig. 7: Overview of *FEATHER* architecture. The compute pipeline (NEST→BIRRD→OB→QM) reads iActs from StaB Ping (or Pong) and writes oActs to StaB Pong (or Ping) **with a new data layout**.

(§IV), (iii) a tool called LayoutLoop for dataflow and layout co-exploration (§V). *FEATHER* provides two specific benefits over prior work in data reordering: (i) supporting arbitrary reorder, and (ii) proposing RIR to hide reordering latency behind computation, and minimize bank conflicts.

III. *FEATHER* Overview

In this section, we provide an overview of *FEATHER* architecture in Fig. 7 and its micro-architectures in Fig. 8.

A. FEATHER's Neural Engine – NEST

Accelerators typically use tens of thousands of PEs organized in 1D arrays like MAERI [35] and SIGMA [42], or 2D arrays like Google TPUv4 [26] and Meta's MTIA [19]. 2D PE arrays have better scalability but are limited in their dataflow options due to their rigid structure, leading to suboptimal utilization due to mismatch of layer shapes and array aspect ratios, as prior works have shown [35], [45]. 1D arrays with flexible distribution and reduction NoCs [32] have been shown to support arbitrary dataflows with full-range of TOPS (§II-A), specifically flexible parallelism and shape. However, they suffer from scalability issues due to their all-to-all NoCs.

This work tries to marry the best of both styles. We find that the all-to-all reduction networks in prior works [35], [42] come with prohibitive resource overheads because of redundant reduction paths. This is to accommodate *arbitrary sized reductions*. In contrast, *FEATHER*'s Neural Engine enables all rows of the 2D PE array to share the same reduction network in a time-multiplexing manner (thereby reducing its cost), without compromising flexibility, throughput, or utilization.

Specifically, *FEATHER*'s **N**eural **E**ngine with **S**patial forwarding and **T**emporal reduction (NEST) works in two phases. One walk-through example for convolution is shown in Fig. 9.

Phase 1: Local Temporal Reduction. *NEST* involves local registers in each PE for temporal (local) reduction of partial sums. This is then followed by a phase of global reduction via the reduction network (described in §III-B).

Phase 2: Interleaved Spatial Forwarding and Reduction. However, unlike prior works where all PEs participate simultaneously in the spatial reduction, the PE rows in *FEATHER* perform spatial reduction one after another, temporally multiplexing on the reduction network. Further, while each PE

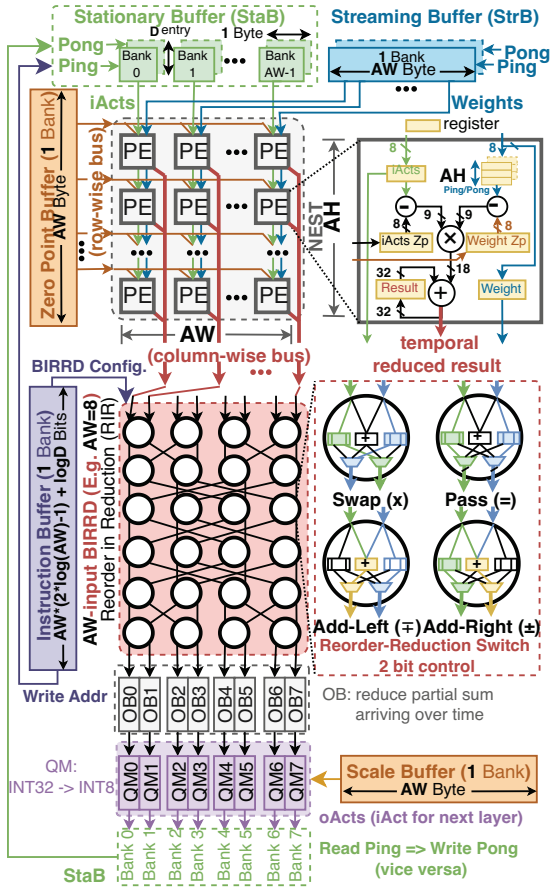


Fig. 8: Micro-architecture of *FEATHER*'s datapath for convolution/GEMM. For convolution, the NEST reads iActs from StaB and weights from StrB, streaming both in a top-to-bottom pipeline. PEs in a column time-multiplex a common output bus. *BIRRD* conducts global spatial reduction and reorders results for targeted StaB banks during reduction, altering data layout in StaB. NEST facilitates inter-layer pipelining by reading iActs from StaB Ping (or Pong) and writes oActs (next-layer iActs) back to StaB Pong (or Ping). Note: *FEATHER* is scalable architecture and we show 8-input *BIRRD* as an example.

row sends its locally reduced results to the reduction network, PEs in other rows continue computation and reduction locally. This is ensured via a pipelining mechanism that guarantees that each row performs *AH* number of local reductions, before participating in the global reduction.

Flexible Dataflow: *FEATHER* retains the ability to support arbitrary dataflow parallelism strategies and shapes (§II-A). This is because Phase 2 can be configured to create arbitrary-sized reduction groups (i.e., all outputs can be unique or any combinations can be reduced) enhancing mapping flexibility.

FEATHER supports inter-layer pipelining. We deploy distinct computation engines for ReLU, BatchNorm, and MaxPooling. For AvgPooling layers, they are transformed into convolution operations and executed within the *NEST*. When there is a sole requirement for reorder and reduction, the PE Array can be bypassed, directing inputs from *NEST* directly to the *BIRRD*. To optimize storage utilization and reduce data movement costs, all computation engines utilize the same on-chip storage.

B. FEATHER's Reordering/Reduction Network – BIRRD

The Butterfly Interconnect for Reduction and Reordering in Dataflows (*BIRRD*) is a multi-stage network designed to reorganize data during the reduction phase. It receives computation results from the previous stage and directs them to new positions in the output buffer while concurrently reducing the data. This process aligns the data in the format needed for the subsequent dataflow, enabling *FEATHER* to seamlessly co-switch (dataflow, layout) for each layer.

Algorithm 1: Inter-stage Connectivity for *AW*-input *BIRRD*

output[i][id]/input[i][id] ($id \in [0, AW)$) refers to id-th output/input port of *BIRRD* switches at the stage i.
FUNCTION reverse_bits(data, bit_range)
  mask = (1≪bit_range) - 1
  reversed_bits = 0
  for i FROM 0 TO bit_range - 1
    if (data (1≪i))
      reversed_bits |= (1≪(bit_range - 1 - i))
  return (data & ∼mask) | reversed_bits
- - - - - - - - - - - - - - - - -
for i in [0, $2 \times log_2(AW)$) // i is stage_id
  for j in [0, AW) // j is port_id
    output[i][j]-input[$i+1$][reverse_bits(j, min($log_2(AW), 2+i, 2 \times log2(AW) - i$))] (- indicates output connects to input)

1) BIRRD Topology: The *BIRRD* topology is interfaced with *NEST* engine one side and output buffer on the other side, and is composed of two butterfly networks back-to-back with $log(AW)$-bit bit reverse connections [16]. This topology grants symmetry with respect to the middle, enabling the construction of each half separately. Each input of *BIRRD* receives data from one column-wise bus of the *NEST* while each output of *BIRRD* forwards the result to one output buffer and eventually back to one bank of stationary buffer (StaB, refer to Fig. 7) . For *NEST* with *AW* columns in total (*AW* must be a power of 2), the *BIRRD* encompasses $2 \times log(AW)$ stages[1] with $AW/2$ switches located at every stage. The inter-stage connections of *BIRRD* are outlined in Alg. 1.

The topology of *BIRRD* has been proven to be strictly non-blocking for unicast (any single data point among concurrent inputs sent to a single output) [5] and rearrangeably non-blocking for multicasting (at least one data point among all concurrent inputs sent to multiple output ports) [8], [16], [36]. We found no multicasting case that it cannot accommodate.

2) BIRRD Reorder-Reduction Switch: The *BIRRD* is built on 2-input×2-output switch (which we call *Egg*) with adder as shown in Fig. 8. Each *Egg* is governed by a 2-bit

[1]4-input *BIRRD* is a special case with only $2 \times log(AW) - 1 = 3$ stages, i.e. the last stages of two half butterfly networks get merged into a single stage.

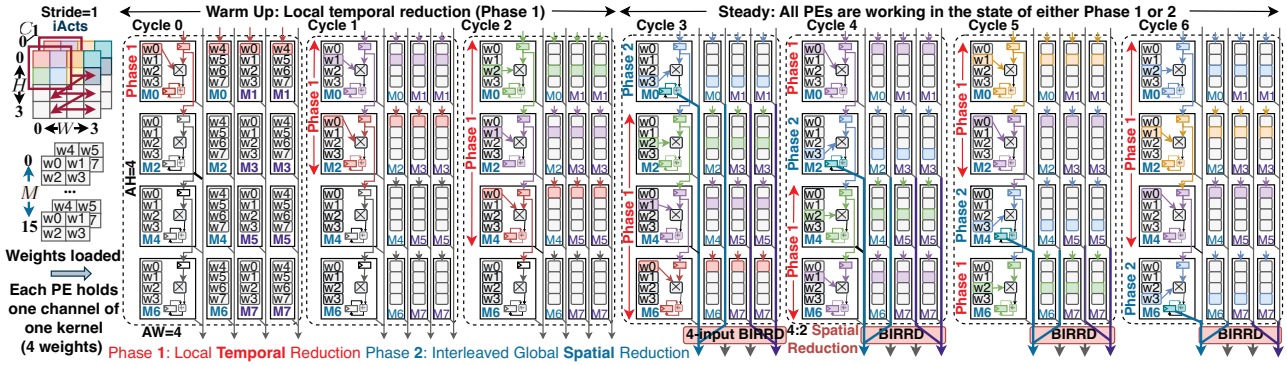


Fig. 9: Illustration of the *FEATHER* with *NEST* and *BIRRD* employing a convolutional operation with a 2×2 weights featuring 2 input channels ($C = 2$) and generating 16 output channels ($M = 16$) across a 4×4 iAct with 2 input channels. The depicted dataflow utilizes a weight-stationary approach, where each PE has a local register file containing a channel of weights (2×2). The dataflow is parallelized for two input channel and two output channel across four PE columns, and for four kernels across four PE rows. In each row, four PEs generate 4 partial sums, contributing to 2 final sums, which thus necessitates a 4:2 spatial reduction in the *BIRRD* to produce two outputs. We assume the weights are already preloaded into *NEST* before the first cycle in this illustration. The iActs are streamed from the top, undergo multiplication with corresponding weight values (e.g., $w0$ in the top-left PE at cycle-0), and are locally accumulated for the next set of inputs (e.g., until cycle-3 in the top-left PE). Following this initial phase of local temporal reduction, the top row transmits the locally reduced result to the *BIRRD* for the second phase of spatial reduction. In the steady state, *BIRRD* reduces data from one *NEST* row per cycle (cycles 4-6). *In steady state, all PEs are working and there is no output bus conflict for PEs of the same column. This is because, during phase-2 of spatial reduction in one PE, remaining PEs of the same column perform local reduction.* In general, $AW \times AH$ *NEST* takes AH^2 cycles to load weights, and ping-pong local registers are instantiated to hide such latency behind computation. *BIRRD* could reduce results from PEs at different rows as long as only one PE per column uses the output bus. **Takeaway:** NEST utilizes local temporal and global spatial reduction to (i) ensure all PEs of the same column share the same output bus without competition while achieving full utilization, and (ii) hide weight loading latency in steady phase.

configuration word, allowing for control of four reorder-in-reduction functionalities (shown in Fig. 8) as follows.

- **Pass (=) / Swap ($\times$)**: directly pass left (right) input data to left (right) output port, or swap them.
- **Add-Left ($\mp$) / Add-Right ($\pm$)**: Accumulates data from input ports and transmits results to the left/right output port, with the secondary output inheriting the input from the same direction.

Extra broadcast functions could be added in the Eggs to duplicate accumulated results in multiple banks of StaB.

3) BIRRD Capability and Routing: *BIRRD* supports

- *Arbitrary Reduction*: We define "reduction group" as a group of inputs that get reduced into one output. *AW*-input *BIRRD* supports arbitrary number of reduction groups (up to *AW*).
- *Arbitrary Reordering*: The rearrangeably multicasting capability enables *BIRRD* to route results from many reduction groups to many arbitrary output ports concurrently.

The examples of *BIRRD* supporting various reordering and reduction patterns are shown in Fig. 10.

From a routing perspective, *reduction can be viewed as a reverse multicasting operation*, where multiple input data points target the same output port and are reduced upon encountering each other at *BIRRD* Eggs. Thus, we adopt the multicasting routing algorithm [4] to establish paths and configurations for *BIRRD* Eggs, enabling reordering during reduction. If a certain input-output connection cannot be established by the algorithm [4], we will brute force all possible configurations. Fig. 10 showcases how *BIRRD* supports arbitrary dataflows and layout switching requirements.

4) Microarchitectural Benefits of BIRRD: Generally, distribution networks like Benes in SIGMA [42] or fat-tree in MAERI [35] necessitate unicast or multicast capabilities to direct data from relevant on-chip buffer banks to specific processing elements (PEs). This necessity becomes obsolete with *BIRRD* (via *RIR*), as it harmonizes data layouts to coincide with dataflows. This enables *FEATHER* to utilize a straightforward point-to-point connection to the input ports of *NEST* without sacrificing flexibility. Consequently, *BIRRD* simplifies the requirements for distribution networks in accelerators, thereby minimizing control, resource, and latency expenses.

C. On-chip Storage and Post-processing

On-chip storage is physically divided into separate buffers with different organizations for concordance with dataflows.

1) Stationary (StaB) and Streaming Buffer (StrB): The typical paradigm of processing convolution or GEMM will keep one type of data stationary, termed a stationary tensor, and stream the other type of data, termed a streaming tensor. *FEATHER* fetches and processes the streaming tensor in the tile granularity. Both StaB and StrB implement a ping-pong

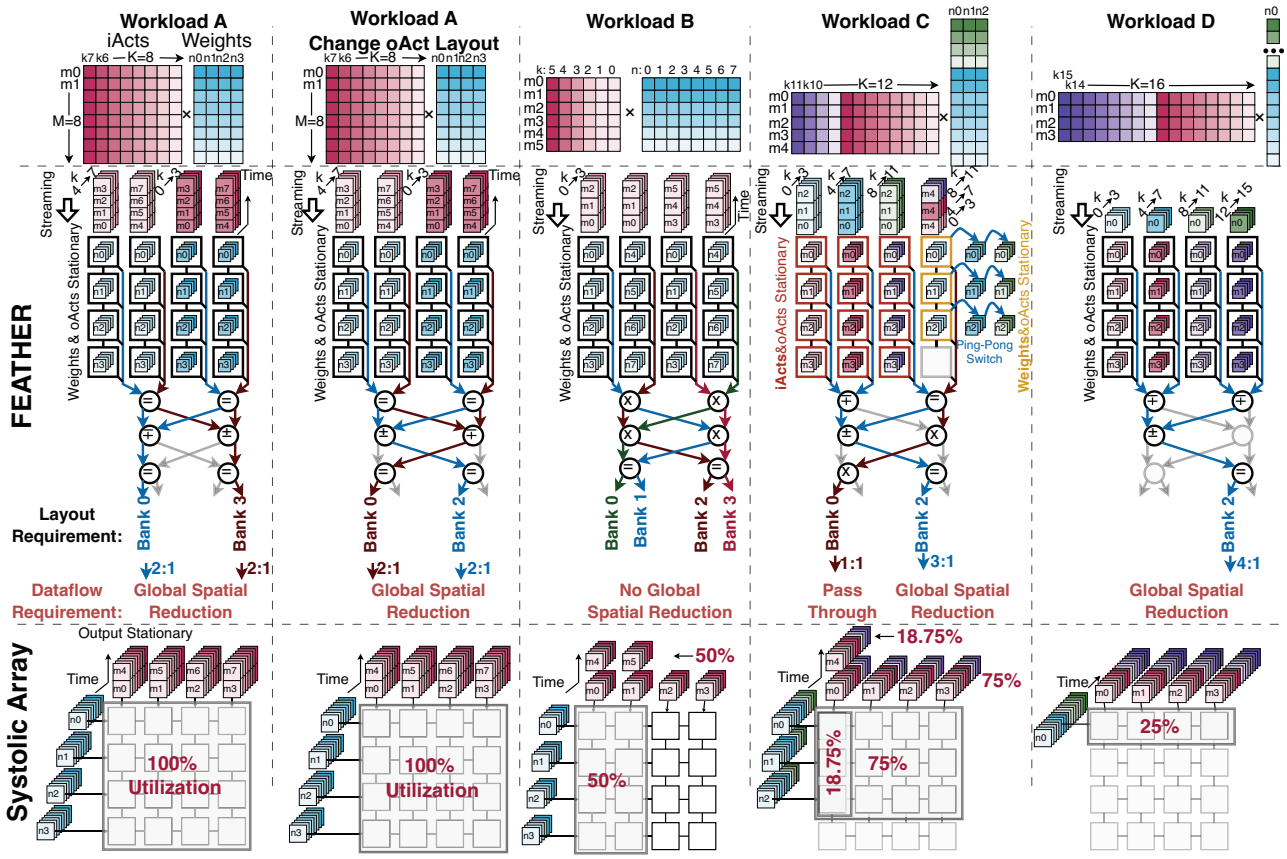


Fig. 10: Comparison between per-layer flexible dataflows in *FEATHER* and fixed-dataflow in the systolic array under GEMM. *FEATHER* dynamically alters layout by redirecting oActs to various banks with distinct writing addresses, exemplified by rerouting a blue result from bank 0 (Workload A) to bank 2 (Workload A Change oAct Layout). *FEATHER* consistently outperforms SA in irregular-sized GEMM (Workload B, C, D), achieving near full utilization. Enhanced utilization arises from (1) enabling cross-column spatial reduction using *BIRRD* in *FEATHER*, e.g. *FEATHER* maps K dimension across the entire 2D array instead of a single PE in SA under workload D. (2) Eliminating SA's horizontal rigid reuse links, thereby enabling independent mappings across columns, e.g. (Workload C) adopting iAct stationary in first three columns and weights stationary in the last column. *BIRRD* could perform pure reordering to change the layout when no spatial reduction is required (e.g. *BIRRD* reordering all incoming results to target banks directly under workload B). **Takeaway:** BIRRD's flexible reduction enhances compute utilization across diverse skewed shapes, expanding the range of dataflows that NEST can efficiently support.

buffer to enable (1) the latency hiding of fetching the next tile from off-chip DRAM, and (2) on-chip inter-layer pipelining.

As for convolution/GEMM (Fig. 8), iActs are kept stationary within StaB Ping (or Pong), and the resulting oActs are written back into StaB Pong (or Ping) with a new layout. Meanwhile, weights are streamed via StrB (Ping/Pong). StaB requires a multi-bank organization (*AW* banks), with each bank storing a single data piece, to accommodate the varied write addresses in different banks necessitated by layout changes in *FEATHER*. Conversely, StrB adopts a simplified single-bank structure with an *AW*-data bandwidth to conserve area, because weights do not need layout reordering.

2) Instruction Buffer (IB): The configurations for *BIRRD* are generated offline and get fetched into IB to configure the reduction networks at run-time.

3) Output Buffer (OB): enables in-situ temporal reduction of partial sums when the reduction size of workloads exceeds the overall reduction capacity of both *NEST* and *BIRRD*. OB has *AW* banks, and each equipped with a 32-bit adder.

4) ZP/Scale Buffer and Quantization Module (QM): employing quantization schemes from PyTorch FBGEMM [31] and QNNPACK [17], with 8-bit zero points and 32-bit scales (housed in ZP/Scale Buffer). The quantization module rescaled down 32-bit oActs and then quantized to 8-bit oActs.

IV. *FEATHER* IN ACTION

In this section, we first showcase one example (Fig. 11) of how *FEATHER* leverages *RIR* to resolve bank conflicts mentioned in Fig. 6b when co-switching dataflow-layout. Then we deep dive into how *FEATHER* enables general layout transformations without bank conflicts through two insights.

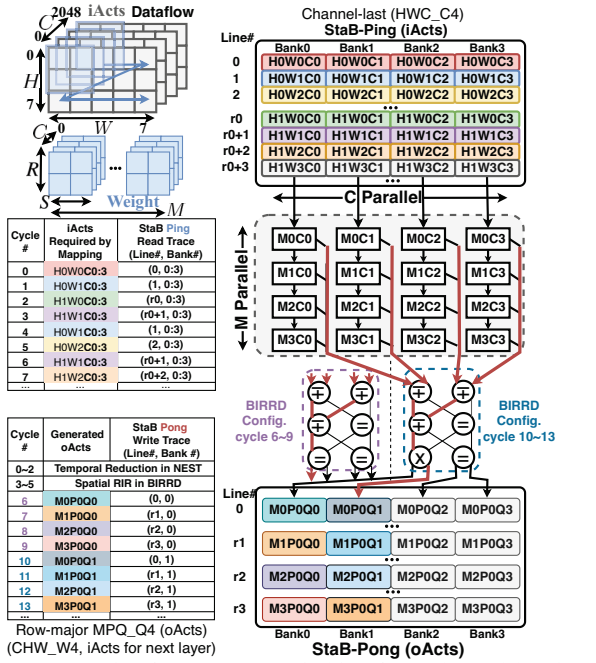


Fig. 11: Example of *FEATHER* switching from channel-last layout (HWC_C4) to a row-major format ($MPQ\_Q4(CHW\_W4)$) during reduction without incurring bank conflicts. This is because multiple iActs are reduced into fewer oActs, thereby reducing accesses within each bank. In this example, *NEST* leverages parallelism along the kernel M and channel C dimensions, reading and vertically streaming four iActs of four input channels from top to bottom. Specifically, at cycle 0, *NEST* fetches $H0W0C0:3$ from (line 0, banks $0:3$), as recorded in the StaB Ping read trace. Subsequent cycles involve a two-stage reduction: temporal reduction within the PE for cycles 0 to 2, and spatial reduction within *BIRRD* for cycles 3 to 5, culminating in a single oAct $M0P0Q0$. This oAct is reordered to bank 0 during reduction and written to line 0 in the StaB Pong during cycle 6. *FEATHER*'s pipelined processing of following iActs is further exemplified in the read/write trace. $M0:3P0Q0$ target bank 0 and use connectivity of *BIRRD* as shown in the left while $M0:3P0Q1$ use the right. For brevity, the notation of $R0:1S0:1$ is omitted, which indicates that each PE in *NEST* holds four weights of one channel. **Takeaway:** FEATHER reorders oActs into next layer's desirable layout during reduction, enabling dataflow/layout co-switching.

A. RIR for Bank Conflicts Mitigation and Layout Transform

In the example shown in Fig. 11, the layout conversion from iActs to oActs is realized via *RIR*, thereby avoiding the explicit latency in reorder after reduction. This efficiency stems from the key insight that *RIR reorders post-reduction oActs into a new layout, rather than directly transforming iActs from one layout to another*.

Specifically, in the reduction phase, numerous iActs naturally get accumulated into fewer oActs and consequently target fewer banks. For example, four iActs get accumulated to one oAct that targets a single line in Fig. 11. Conversely, if we directly transform the layout of iActs from channel-last to row-major, four iActs (H0W0C0:3) would target four different lines within the same bank under row-major layout, leading to bank conflicts.

B. (Dataflow, Layout) Flexibility for Bank Conflicts Eradication

While the strategy of 'reordering post-reduction oActs' aids in reducing bank conflicts, conflicts may still arise when the number of partial sums to write into memory exceeds the number of writing ports of the memory. This scenario is particularly common in scaled-up 128×128 compute array (Google TPUv4 [26]), as it generates more oActs concurrently.

FEATHER fully eliminates conflicts with the second key insight that *FEATHER picks the dataflow with the number of oActs (partial sums) matching with the number of memory write ports.* In essence, *FEATHER* employs dataflows free from bank conflicts, and the flexible reduction of the *BIRRD* consistently allows *FEATHER* to identify such dataflows with high performance and efficiency.

In summary, *RIR* together with flexible dataflows selection enable *FEATHER* to switch among arbitrary layouts without incurring bank conflicts.

V. Layoutloop

FEATHER enables (dataflow, layout) co-switching at the layer granularity to achieve optimal latency and energy efficiency. However, deciding which (dataflow, layout) to use for *FEATHER* is not trivial because both dataflow and layout have huge space, e.g. $10^{36} \times 10^{8}$ for a single convolution layer (ResNet-50 layer 1) [27][2], necessitating systematic exploration. For this aim, we enhance Timeloop [41], a state-of-the-art dataflow search framework with (1) physical storage modeling and (2) systematic layout assessment capabilities, and term it as Layoutloop to distinguish it from native Timeloop. We employ Layoutloop to explore dataflows under various layouts for *FEATHER*, selecting the dataflow-layout pair that minimizes energy delay product for each layer.

A. Physical Storage Modeling

Layoutloop models physical storage as ($num\_line \times line\_size$) 2D array with "$conflict\_depth$" specifying number of lines in each bank with the following reasoning.

Bank Organizations: Current storage uses diverse organizations, including 2D/3D with various groupings. Managing these disparate physical organizations can be complex. However, as storage is usually accessed line by line (or block), we can abstract different organizations into a logical $num\_line \times line\_size$ 2D array. This abstraction allows layout modeling to handle these 2D abstract arrays directly, retaining generality without dealing with specific physical organizations.

Bank Port Constraints: Storage comes with an inherent limitation of the total number of ports in each bank. Concurrent

[2]A flattening of 4 iActs dimensions $(N = 1, C = 3, H = 224, W = 224)$ into two nested loop (Fig. 3) introduces $8! = 40320$ order possibilities and $(1, 2, 16, 16)$ factorization possibility. The product leads to 10^8 layout choices.

read/write operations exceeding available read/write ports lead to bank conflicts. Thus, $conflict\_depth$ is utilized to denote the total number of lines within a single bank.

B. Bank Conflicts Assessment

Layoutloop models slowdown by judging whether bank conflicts occur when analyzing data access to the on-chip buffer with a specific layout. A $\max(N_P/N_L, 1)$ slowdown is introduced if N_L lines are accessed from a bank with N_P ports. Finally, we also modify Timeloop's mapper to consider data layout during dataflow search.

VI. Evaluation

A. Methodology

We implement *FEATHER* in Verilog and Xilinx HLS. Verilog-based implementation delivers precise micro-architecture design while HLS-based implementation enables the native usage of Xilinx IPs for buffer, control, and peripherals for better end-to-end performance on Xilinx FPGAs. We evaluate its resources on TSMC 28 nm high performance technology node using the Verilog-based implementation. We compare its end-to-end wall-clock latency against SoTAs with open-sourced end-to-end implementations on real FPGA devices. We also model *FEATHER* in Layoutloop (§V), including energy overheads, to compare it against SoTA accelerators that do not have open-sourced end-to-end deployable codes. Tab. IV summarizes our evaluation setup.

1) Baselines and Workloads:

Baselines for real-device evaluations. We compare *FEATHER* against Xilinx DPU [2] , Gemmini [21], and Edge TPU [46], as they can be deployed in an end-to-end fashion. *FEATHER* and Xilinx DPU [2] are deployed on the same Xilinx ZCU 104 FPGA board. While Gemmini is deployed on AWS-F1 FPGA server [21] using FireSim to emulate its per-layer processing latency. Edge TPU [46] runs on a USB accelerator [1] attached to a Raspberry Pi 4B. As for all four designs, we normalize throughput by the number of PEs (i.e., MAC units) and clock frequency [3] for a fair comparison.

Baselines for Layoutloop. *FEATHER* is further compared against NVDLA [39], Eyeriss [14] and SIGMA [42] in Layoutloop. Detailed modifications/specs are listed in Tab. IV

Workload. BERT (representative of cloud workloads); ResNet-50 and MobiletNet-V3 (Mob-V3) as edge workloads.

2) FEATHER Dataflow/Layout Setup:

- **Search Space.** Dataflow design space is constructed by arbitrary nested loops as shown in Fig. 1. We use layout patterns used by prior accelerators [43] as layout space [4].
- **Searching Algorithm.** We exhaustively search layout space for global optimal. To find optimized dataflows, we use Timeloop's internal hybrid search algorithm (exhaustive + search-space pruning). Recent works [29] show that its results are comparable to sophisticated search methods [22], [23], [27] but is slower in wall clock time. We ran the search with multiple threads constrained on search size and victory conditions.
- **Performance Metric.** We use Energy-Delay-Product (EDP) as the performance metric for a dataflow/layout pair.
- **Overall Search Flow.** Dataflow/layout cosearch is conducted for each layer independently. The optimal dataflow-layout pair with the best EDP is chosen for each layer of ResNet-50 and Mob-V3 in the Layoutloop evaluation. For end-to-end FPGA deployment of ResNet-50, we simplify engineering efforts by selecting the two layouts with the best latency and energy efficiency on DepthWise Conv. and typical Conv., and enable *FEATHER* to switch between them per layer.

[3]Both GEMMINI and FEATHER could run at 1 GHz under TSMC 28 nm ASIC flow. However, the parallel simulation synthesis toolchain of firesim limits GEMMINI's clock frequency to 50 MHz on AWS's f1.2xlarge FPGA.

[4]Conv: HWC_C32, HWC_W32, HWC_H32, HWC_C4W8, HWC_C4H8, HWC_W4H8, HWC_C4W4H2; GEMM: we note input/weights/output as $M \times K/N \times K/M \times N$ with inputs layout as MK_K32, MK_M32, MK_M4K8.

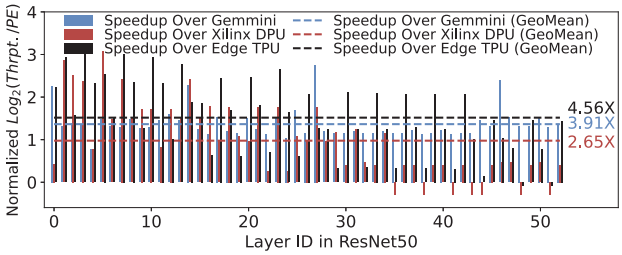


Fig. 12: FEATHER vs. SoTAs on real devices. We run each layer for 100 times to obtain average layer latency, and then normalize throughput by number of PE and clock frequency.

B. End-to-end Real-device Latency Evaluation

1) FEATHER vs. Gemmini: *FEATHER* achieves a 3.91× geomean normalized throughput improvement than Gemmini as shown in Fig. 12, as Gemmini adopts a fixed dataflow (weights stationary with degree of parallelism being 16 in both C and M), leading to under-utilization when C of workload is not divisible by 16. The flexibility of *FEATHER* in the parallelism of M,C,H,W delivers its performance improvement.

2) FEATHER vs. Xilinx DPU: The 2.65× more throughput of *FEATHER* over Xilinx DPU stems from the low steady-state utilization of Xilinx DPU under convolution 3×3 (75% utilization), 7×7 (21.8∼87.5% utilization), and *FEATHER* pushes both utilization to 100% and 90.4% in the steady state. This is because Xilinx's DPU with 1152 PEs only supports a single dataflow with parallelism $(12, 12, 8)$ in $(M, C, H/W)$. In deep layers with a large number of input channels (C) and kernels (M), both *FEATHER* and Xilinx DPU achieve a steady utilization of 100%. However, Xilinx DPU outperforms *FEATHER* for these layers as our controller is not as optimized as DPU's (an engineering optimization part of our future work).

3) FEATHER vs. Edge TPU: 4.91× speedup comes from flexibility of FEATHER in dataflow and layout.

C. Layoutloop-based Latency/Energy Evaluation

Latency and energy efficiency are affected by (1) compute utilization determined by dataflows, and (2) effective memory bandwidth considering bank conflicts.

TABLE IV: Evaluation setup for SoTAs and *FEATHER* (→ indicates the modifications from the original design).

	Run-time Flexibility in (Layout, Dataflow②, Reorder)	(#PE,	on-chip BW bit/cycle,	DataType)	Area (μm^2)/Clock Frequency	Evaluation Method Real-device/Layoutloop
Edge TPU	(fix , T, none)	(1024,	unknown,	int8)	500 MHz (ASIC)	Coral USB accelerator [1]
Xilinx DPU	(fix , T, none)	(1152,	unknown,	int8)	100 MHz (FPGA)	end-to-end on ZCU104
Gemmini	(fix, T, none)	(1024,	512,	int8)	50 MHz① (FPGA)	FireSim on AWS EC2 F1
FEATHER	(flexible, TOPS, Reorder In Reduction)	(1296,	720,	int8)	100 MHz (FPGA)	end-to-end on ZCU104
NVDLA-like	(fix, T, none)	(16×16,	25→256,	int8)	808K (TSMC-28, 1 GHz) ⑤	Layoutloop
Eyeriss-like	(fix, TS, none)	(14×12→16×16,	192→256,	int16→int8)	1394K (TSMC-65, 200 MHz)	Layoutloop
SIGMA-like	(fix③, TOPS, none)					
SIGMA-like	(flexible, TOPS, off-chip reordering④)	(65536→256,	256,	bf16→int8)	990K (TSMC-28, 500 MHz)	Layoutloop
Medusa-like	(flexible, TOPS, on-chip line rotation)					
TPU-like	(flexible, TO, transpose/row reorder)	(8×"128×128",	256,	int8)	600K (7nm, 1050 MHz)	Layoutloop
MTIA-like	(flexible, TOP, transpose)	(64×"32×32",	1024,	int8)	373K (TSMC-7, 800 MHz)	Layoutloop
FEATHER	(flexible, TOPS, *RIR*)	(16×16,	8000,	int8)	338K (TSMC-28, 500 MHz)	Layoutloop

①: Latency scaled to 100 MHz. We standardized the frequency at 100 MHz just for a fair comparison purpose, which is not indicative of the maximum clock frequency achievable on ASIC implementations.; ②: Terminology defined in §II-A; ③: HWC_C4W8 or HWC_C32; ④: off-chip bandwidth=128 GB/s. ⑤: compute area only.

Label	Layout Setup compile	runtime
NVDLA-like	fixed	fixed
Eyeriss-like	fixed	fixed
SIGMA-like	flex	fixed
Medusa-like	line rotation (Fig. 5b)	
MTIA-like	transpose (Fig. 5c)	
TPU-like	transpose + row reorder (Fig. 5d)	
FEATHER	arbitrary reorder (Fig. 5e)	

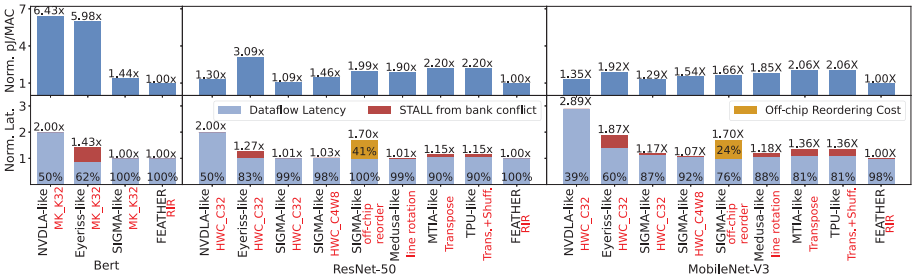


Fig. 13: *FEATHER* vs. SoTA using Layoutloop (Percentage inside each blue bar indicates average steady-state PE utilization. Red bar indicates bank conflict slowdown, while yellow bar indicates off-chip reordering costs. Lower is better. (The red text in the x-axis of the right chart mentions the fixed layout or the layout reordering mechanism for each design.)

With per-layer dataflow-layout switching, *FEATHER* achieves the peak steady-state utilization of 100%, 100%, and 98.3% with zero bank conflict slowdown under BERT, ResNet-50, and Mob-V3, separately. This indicates that FEATHER consistently provides desirable layout for all three workloads.

1) FEATHER vs. NVDLA: *FEATHER* achieves 2×/2×/2.89× speedup and 6.43×/1.3×/1.35× higher efficiency over NVDLA under BERT/ResNet-50/Mob-V3. Leveraging fixed weights/output stationary dataflows under a fixed HWC_C32 layout, NVDLA will not encounter any bank conflicts, which delivers good energy efficiency. However, NVDLA only allows flexible tiling sizes T (definition in §II-A), and such dataflows suffer from low utilization (50% and 39%), explaining higher normalized latency in Fig. 13.

2) FEATHER vs. Eyeriss: The 1.43×/1.27×/1.87× speedup and 5.98×/3.09×/1.92× higher efficiency of *FEATHER* over Eyeriss comes from both bank conflicts elimination and higher-performance dataflows. Specifically, Eyeriss adopts row-stationary dataflow and enables flexible tiling and shape but such sub-flexibility comes with the price of bank conflicts. Compared against Eyeriss-like, *FEATHER* incurs 6% more area by introducing *BIRRD* and controller, as shown in Fig. 14b.

3) FEATHER vs. SIGMA: SIGMA supports flexibility in all 4 dimensions of dataflows (§II-A), but does not have reordering capability (§II-E2). We thus evaluate static layout, off-chip and three on-chip reordering scenarios, as illustrated in Fig. 6.

- **Fixed Layout + No Reordering:** SIGMA adopts a fixed layout at runtime and keeps iActs and oActs always on-chip. And we leverage Layoutloop to search dataflows with minimal bank conflicts. Results under two layouts (HWC_C32 and HWC_C4W8) out of seven layouts delivering relatively better latency and energy efficiency are depicted in Fig. 13. Proposed Layoutloop could fully utilize SIGMA's flexibility in TOPS to identify dataflows with fewer bank conflicts, resulting in a small speedup of *FEATHER* over SIGMA (the same for BERT, 1.01×/1.03× for ResNet-50, and 1.17×/1.07× for Mob-V3). Although Layoutloop could identify dataflows for SIGMA with good latency, such dataflows always need to read more lines than dataflows adopted by *FEATHER*, resulting in the high energy efficiency of *FEATHER* (1.44× for BERT, 1.09×/1.46× for ResNet-50, and 1.29×/1.54× for Mob-V3).
- **Concordant Layout + Off-chip Reordering:** SIGMA pays the latency and energy costs to send oActs back off-chip and changes layout per layer. Such reordering cost is explicitly shown in Fig. 13. In the case of high compute-intensive ResNet-50, the latency of off-chip reordering could be almost hidden behind computation latency when adopting HBM with 128 GB/s, leading to the pure energy costs of moving data back and forth between HBM and compute. By contrast, in low compute-intensive MobV3, off-chip reordering exposes 24% critical latency, which further restricts the performance of dataflows as SIGMA has to use some dataflows with the least off-chip accesses. This explains the 1.7×/1.7× speedup and 1.99×/1.66× efficiency improvement of *FEATHER*.
- **Flexible Layout + On-chip line Rotation:** SIGMA is equipped with line rotation, proposed in Medusa [48], to mitigate bank conflicts when reading three lines from the same bank. But typical workloads often access more than 4 lines per cycle. Further, all seven on-chip layouts utilized in the paper require word-granularity data reordering to switch

from one to the other, a capability supported by *FEATHER* but not line rotation, which explains the 1.01×/1.18× speedup and 1.90×/1.85× efficiency improvement of *FEATHER*.

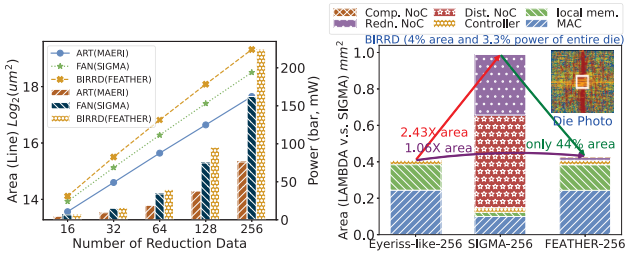
- **Flexible Layout + On-chip Transpose (MTIA-like):** We enhance SIGMA with on-chip data transpose (Fig. 5c), the reordering capability provided by MTIA and TPUv4. Transpose is effective for reducing bank conflicts caused by single-dimensional parallelism. However, multi-dimensional parallelism based bank conflicts require finer-grain data reordering, a function supported by *RIR* but not by transpose. Therefore, FEATHER demonstrates speedups of 1.15×/1.36× and achieves 2.2×/2.06× greater efficiency, highlighting its superior handling of complex data layout transformations.
- **Flexible Layout + On-chip Transpose and Row-reorder (TPU-like):** On top of MTIA-like, we further add row-reorder (Fig. 5d). Yet, this enhancement does not reduce on-chip buffer accesses nor modify data locations compared to transpose alone, resulting in no further latency saving or efficiency gains.

D. Resources and Timing Evaluation under TSMC 28nm

1) BIRRD vs. FAN [35]/ART [42]: We have implemented *BIRRD* in Verilog, and obtained its post-layout resources under different scales. Fig. 14a provides a comparative evaluation of *BIRRD* with other reduction networks like SIGMA's FAN and MAERI's ART, considering int32 adders. Here, the area is represented by lines while the bars demonstrate power consumption. An AW-input *BIRRD* has more stages ($2log_2(AW)$) than FAN/ART ($log_2(AW)-1$). Consequently, *BIRRD* consumes about 1.43×/2.21× more area and 1.17×/2.07× more power than FAN/ART. Despite these overheads, the adoption of *BIRRD* is justifiable for two primary reasons:

- Unlike FAN/ART which requires one $AW \times AH$-input instance for all 1D PEs, a single AW-input *BIRRD* instance can fulfill the reordering and reduction needs for all 2D PEs. As a result, when integrated into *FEATHER*, *BIRRD* achieves a resource-saving of 94% over the FAN in SIGMA, as indicated by the Reduction NoC (Redn. NoC in Fig. 14b).
- While both MAERI and SIGMA necessitate a complex distribution NoC such as fat-tree, crossbar, or Benes, *BIRRD* eliminates such requirements as data always come in a perfect layout without further redistribution demands (§III-B4). Thus *FEATHER* replaces distribution NoC with pt-to-pt connections.

2) FEATHER vs. SIGMA/NVDLA: The combined simplification of the distribution NoC and implementation of a singular *BIRRD* instance results in a substantial 2.93× resource reduction of SIGMA - for *FEATHER* with an equal number of 256 PEs, as illustrated in Fig. 14b. *FEATHER* has large local memory as each PE in $AW \times AH$ *FEATHER* needs to keep sufficient data inside local memory to perform local reduction when other PE rows are using oAct buses. Further, *FEATHER* adopts 2D PE array with better scalability compared with 1D PE design in SIGMA. We further implement a NVDLA-like 1D PE array serving as a fix-dataflow baseline.



(a) Reduction Network. (b) Resources Breakdown.

Fig. 14: ASIC resource comparison (*FEATHER* vs. SoTA). 16×16 *FEATHER* place-and-route at TSMC 28nm.

E. Timing Analysis

We layout *FEATHER* with 64, 256, and 1024 PEs, requiring *BIRRD* with 8, 16, and 32 inputs. The die photo of *FEATHER* with 16×16 PEs is shown in Fig. 14b revealing that *BIRRD* consumes only 4% of the overall post-layout area in the TSMC 28nm process. *BIRRD* does not have long wires because it is placed outside the PE array as a standalone module instead of spreading among PEs like FAN in SIGMA or ART in MAERI. *BIRRD* attains a peak clock frequency of 1.5 GHz across all scales. The timing critical path of *FEATHER* is the wire connecting local weights registers to 9-bit multiplier in PE, maxing at 1 GHz, similar to SoTA accelerators [26].

VII. Related Work

DNN Accelerators. Most DNN accelerators today (especially those with end-to-end deployment support [6], [7], [10], [21], [26], [39], [51]) either rely on fixed dataflows - thus fixed layouts or support flexible dataflows [15], [35], [42], [50] but do not consider effects of data layout when switching dataflows, both of which hurt performance. Tab. I,III contrast these.

Layout Reordering Support. To mitigate bank conflicts, Medusa [48] introduces line rotation, which rotates one line inside the conflicted bank into different banks when moving data from off-chip DRAM to on-chip compute. However, typical accelerators with higher parallelism require data in finer word granularity, which leads bank conflicts to line rotation.

To enable word-level layout reordering, MTIA [19] introduces a Memory Layout Unit (MLU) that enables transposing, concatenating, and reshaping with 4/8/16/32-bit data types. Besides these three layout transforms, *FEATHER* further supports arbitrary data reordering layout transformations through *BIRRD*. Moreover, extra reordering latency from MLU is completely hidden behind reduction in *FEATHER* through RIR.

VIII. Conclusion

This work motivates the need for data layout reordering support for switching between dataflows in DNN accelerators. We introduce *FEATHER*, an innovative accelerator incorporating a novel multi-stage reduction networks called *BIRRD* to implement a unique on-chip reordering strategy for reordering data in reduction. This facilitates simultaneous dataflow and layout switching in *FEATHER* without explicit latency costs, resulting in speedup over SoTAs by using less area.

IX. Acknowledgement

We thank Hyoukjun Kwon, Geonhwa Jeong and Raveesh Garg for feedbacks, Angshuman Parashar for concordant layout terminology, Sixu Li for place-and-routing, Yongan Zhang for ZCU104 maintenance. This work was supported in part by ACE, one of seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA.

Appendix

A. Abstract

This artifact contains three different flows to evaluate FEATHER with different fidelity against different baselines: (1) end-to-end evaluation on the realistic ZCU 104 FPGA evaluation board, (2) analytic analysis using the mapping search and evaluation framework, LayoutLoop (details in §V) and (3) Verilog implementation with synthesis and place-and-routing evaluation flow.

B. Artifact check-list (meta-information)

Inside the repo, we provide three different experiments in three separate folders. Each folder consists of (1) pre-run results for each experiments, and (2) detailed step-by-step operation to reproduce the pre-run results.

1) Experiment Set 1 - Fig. 12: deploys *FEATHER* on ZCU 104 FPGA board and evaluate the end-to-end layer-wise latency of processing ResNet-50.

- Pre-run per-layer results for inference convolution 3×3, 1×1 layers of ResNet50 are stored in the output of provided jupyter notebook "feather.ipynb".
- Pre-built FPGA bitstream for running on ZCU104 FPGA.

2) Experiment Set 2 - Fig. 13: automated dataflow-layout co-exploration, leveraging Layoutloop, to investigate performance of various baselines and *FEATHER*.

- LayoutLoop Framework in path "LayoutLoop/layoutloop".
- Configurations for FEATHER (arbitrary layout choice), SIGMA (arbitrary layout choice), SIGMA (off-chip reordering), MTIA-like (Transpose), TPU-like (Transpose + Shift), SIGMA-like (HWC_C4W8), SIGMA-like (HWC_C32), Medusa-like (Line Rotation), Eyeriss-like (HWC_C32), NVDLA-like (HWC_C32), at path "LayoutLoop/configurations".
- Pre-run results at path "LayoutLoop/pre_run_results".
- Dataflow-Layout Co-searching scripts (> 24 hours runtime).

3) Experiment Set 3 - Fig. 14: ASIC synthesis and place-and-route on Verilog implementation of *FEATHER*.

- Pre-run results.
- automated scripts for synthesis and PnR (> 96 hours).

An overview of the dependency is listed as follows.

- **Algorithm**: We adopt pruned random searching algorithm to explore large dataflow design space under various layouts.
- **Program**: We provide three experiment sets with click-and-run program entries.
- **Compilation**: We provide step-by-step compilation guideline in the repo and a pre-built docker with all pre-compiled programs.
- **Binary**: The pre-built hardware binary for realistic FPGA deployment is provided in the repo.
- **Model:** We use ResNet-50 for end-to-end FPGA evaluation, and meta data from ResNet-50, MobileNet-v3 and BERT for LayoutLoop analytic analysis.
- **Run-time environment:** Ubuntu, version does not matter.
- **Hardware:** We provide the access for ZCU 104 evaluation board. Users need to have multi-core CPU with 32 GB memory.
- **Metrics:** Latency, Latency-Energy-Product
- **How much time is needed to prepare workflow (approximately)?:** Estimated (1) 1 minutes (2) 10 mins (3) 10 mins.
- **How much time is needed to complete experiments (approximately)?:** Estimated (1) 10 minutes (2 & 3) > 96 hours.
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** MIT
- **Workflow framework used?:** Our proposed Layoutloop is built upon Timeloop [41]

C. Description

1) How to access: The artifact is available at 10.5281/zenodo.10999154.

2) Hardware dependencies: ZCU104 FPGA evaluation board is required to reproduce the end-to-end evaluation results of FEATHER under ResNet-50.

3) Software dependencies:

- (Experiment-1) Prebuilt PYNQ 3.0.1 image for ZCU104 FPGA board with Python 3.10.2, which is available at http://www.pynq.io/boards.html.
- (Experiment-2) scons, libconfig++-dev, libboost-dev, libboost-iostreams-dev, libboost-serialization-dev, libyaml-cpp-dev, libncurses-dev, libtinfo-dev, libgpm-dev, cmake; Python 3.8 with matplotlib, numpy, pandas.
- (Experiment-3) Synopsys 2022.12-SP5 and Cadence innovus v21.14-s109_1, both could be other versions. TSMC 28nm technology standard cell libraries.

D. Installation

We provide detailed installation guide and step-by-step instructions to reproduce the results in the repository.

1) Results Visualization: Our visualization requires the following python packages.

```
$ git clone <repo_url>
$ conda create -n <favoriate_name> python=3.8
$ conda activate <favoriate_name>
$ pip3 install matplotlib numpy pandas
$ python results_generation.py
```

2) LayoutLoop Setup: The compilation and code base requires following libraries on Ubuntu system.

```
$ sudo apt install scons libconfig++-dev \
libboost-dev libboost-iostreams-dev \
libtinfo-dev libboost-serialization-dev cmake \
libyaml-cpp-dev libncurses-dev libgpm-dev
```

We also provide pre-built docker at https://tinyurl.com/layoutloop. Install the following packages to run the docker.

```
$ sudo apt-get install docker-ce containerd.io \
docker-ce-cli  docker-buildx-plugin \
docker-compose-plugin
$docker load -i feather_layoutloop_docker.tar.gz
```

E. Experiment workflow

1) Experiment Set 1, End-to-end Inference on FPGA:

- We require ZCU104 with pre-built PYNQ image, which provides a jupyter notebook portal.
- Copy the provided "feather/feather.ipynb" into the jupyter notebook and then run all blocks.

2) Experiment Set 2, LayoutLoop Analytic Analysis: run the following commands within the pre-built docker. The DSE might take 1 day to finish, depending on the machine.

```
$ docker run -it feather_layoutloop
$ git clone <provided_url>
$ cd FEATHER/LayoutLoop/configurations
$ git pull
$ make clean
$ make conv_dse # DSE for ResNet-50, Mob-v3
$ make gemm_dse # DSE for Bert
```

3) Experiment Set 3, Synthesis and PNR under TSMC 28nm: For synthesis, we use design compiler "dc_shell"

```
<setup environment for synopsys>
$ cd FEATHER/FEATHER_RTL/scripts/
$ source :run_syn
```

For place and routing, we use innovus.

```
<setup environment for innovus>
<Finish Synthesis First>
$ cd FEATHER/FEATHER_RTL/
$ innovus
> source PnR.tcl
```

F. Evaluation and expected results

1) Exp. Set 1: End-to-end latency on FPGA: The layerwise latency of running various models will be shown in the end of the jupyter notebook. We normalize results from different designs using "normalized throughput per PE", where throughput is measured by inverse of latency under single batch. The visualized result is shown in Fig. 12.

2) Exp. Set 2: LayoutLoop Analytic Analysis: Per-layer results from Layoutloop could be found at "LayoutLoop/configurations/results" with following naming pattern.

- design_name_layout_policy_slowdown.csv
- design_name_layout_policy_utilization.csv
- design_name_layout_policy_pj_commpute.csv
- design_name_layout_policy_cycle.csv

We calculate GeoMean of "pJ/compute" and "cycle", and then normalize all results by FEATHER's performance with the visualized results shown as Fig. 13.

3) Exp. Set 3: Synthesis and PNR under TSMC 28nm: The final reports of synthesizing *FEATHER* at a specific scale will be listed in the "reports" folder, including

- feather_top_area.rpt
- feather_top_dw_area.rpt
- feather_top_power.rpt
- feather_top_timing.rpt

The final reports of PnR contain

- area.rpt, which contains Post-PnR area value.
- power.rpt, which contains Post-PnR power value.
- time, timingReports. # Both are timing reports.

TABLE V: Post-PnR *FEATHER* Area/Power at various shapes.

Shape	Area (μm^2)	Power(mW)	Frequency (GHz)
64×128	36920519.69	26400.00	1.00
64×64	18389176.19	13200.00	1.00
32×32	2727906.70	961.70	1.00
16×32	965665.10	655.55	1.00
16×16	475897.19	323.48	1.00
8×8	97976.46	65.25	1.00
4×4	24693.98	16.28	1.00

G. Experiment customization

1) Exp. Set 2: LayoutLoop Analytic Analysis:

Different Configurations: LayoutLoop adopts the same architecture, dataflow constraint and mapper configurations format as TimeLoop with detailed documentations listed at https://timeloop.csail.mit.edu/v4/input-formats/design. Further, we argument LayoutLoop to support the analysis of layouts. The layout definition is shown in §3. The locations of these configurations are listed below.

- architecture design: "FEATHER/LayoutLoop/configurations/arch_designs/"
- dataflow constraints: "FEATHER/LayoutLoop/configurations/arch_designs/systolic_constraint/mapspace.yaml", the dataflow constraint needs to match hierarchies of components in the architecture design.
- mapper: "FEATHER/LayoutLoop/configurations/mapper/"
- Layout: "FEATHER/LayoutLoop/configurations/layout/"

Different on-chip reordering modeling methods are activated by enabling different global macro

- *Transpose:* ENABLE_TRANSPOSE
- *Line Rotation:* MEDUSA

By default, Layoutloop assumes no on-chip reordering.

2) Exp. Set 3: LayoutLoop Analytic Analysis: The provided Verilog implementation of *FEATHER* is a parameterized scalable template, which allows users to change the shape of *FEATHER* by modifying the input parameters at the top module "FEATHER/FEATHER_RTL/RTL/feather_top.v". Users could modify the following parameters into value from $(4, 8, 16, 32, 64)$ to investigate the area and power of *FEATHER* at different scsales.

```
module feather_top #(
    parameter DPE_COL_NUM  = 64,
    parameter DPE_ROW_NUM  = 64,
    ...
```

REFERENCES

[1] "Coral usb accelerator," https://coral.ai/products/accelerator/, accessed: 2024-02-23.

[2] "Xilinx deep learning processing unit," https://docs.xilinx.com/r/1.2-English/ug1414-vitis-ai/Deep-Learning-Processor-Unit-DPU, accessed: 2022-12-10.

[3] W. Ali, S. Abdelkarim, M. Zidan, M. Zahran, and A. El Sallab, "Yolo3d: End-to-end real-time 3d oriented object bounding box detection from lidar point cloud," in *Proceedings of the European Conference on Computer Vision (ECCV) Workshops*, 2018, pp. 0–0.

[4] S. Arora, T. Leighton, and B. Maggs, "On-line algorithms for path selection in a nonblocking network," in *Proceedings of the Twenty-Second Annual ACM Symposium on Theory of Computing*, ser. STOC '90. New York, NY, USA: Association for Computing Machinery, 1990, p. 149–158. [Online]. Available: https://doi.org/10.1145/100216.100232

[5] ——, "On-line algorithms for path selection in a nonblocking network," in *Proceedings of the twenty-second annual ACM symposium on Theory of computing*, 1990, pp. 149–158.

[6] P. Behnam, J. Tong, A. Khare, Y. Chen, Y. Pan, P. Gadikar, A. Bambhaniya, T. Krishna, and A. Tumanov, "Hardware–software co-design for real-time latency–accuracy navigation in tiny machine learning applications," *IEEE Micro*, vol. 43, no. 06, pp. 93–101, nov 2023.

[7] P. Behnam, J. Tong, A. Khare, Y. Chen, Y. Pan, P. Gadikar, A. R. Bambhaniya, T. Krishna, and A. Tumanov, "Subgraph stationary hardware-software inference co-design," 2023.

[8] B. Bogdanski, "Optimized routing for fat-tree topologies," *Department of Informatics, Faculty of Mathematics and Natural Sciences, University of Oslo, Norway*, 2014.

[9] P. Chatarasi, H. Kwon, A. Parashar, M. Pellauer, T. Krishna, and V. Sarkar, "Marvel: A data-centric approach for mapping deep learning operators on spatial accelerators," *ACM Trans. Archit. Code Optim.*, vol. 19, no. 1, dec 2021. [Online]. Available: https://doi.org/10.1145/3485137

[10] P. Chatarasi, S. Neuendorffer, S. Bayliss, K. Vissers, and V. Sarkar, "Vyasa: A high-performance vectorizing compiler for tensor convolutions on the xilinx ai engine," in *2020 IEEE High Performance Extreme Computing Conference (HPEC)*, 2020, pp. 1–10.

[11] K. Chellapilla, S. Puri, and P. Simard, "High performance convolutional neural networks for document processing," in *Tenth international workshop on frontiers in handwriting recognition*. Suvisoft, 2006.

[12] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," *arXiv preprint arXiv:1706.05587*, 2017.

[13] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2016.

[14] ——, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.

[15] Y.-H. Chen, T.-J. Yang, J. Emer, and V. Sze, "Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices," *arXiv preprint arXiv:1807.07928*, 2018.

[16] W. J. Dally and B. P. Towles, *Principles and practices of interconnection networks*. Elsevier, 2004.

[17] M. Dukhan, Y. Wu, and H. Lu, "Qnnpack: Open source library for optimized mobile deep learning," 2018.

[18] V. Fedyunin, "(beta) channels last memory format in pytorch¶." [Online]. Available: https://pytorch.org/tutorials/intermediate/memory_format_tutorial.html

[19] A. Firoozshahian, J. Coburn, R. Levenstein, R. Nattoji, A. Kamath, O. Wu, G. Grewal, H. Aepala, B. Jakka, B. Dreyer, A. Hutchin, U. Diril, K. Nair, E. K. Aredestani, M. Schatz, Y. Hao, R. Komuravelli, K. Ho, S. Abu Asal, J. Shajrawi, K. Quinn, N. Sreedhara, P. Kansal, W. Wei, D. Jayaraman, L. Cheng, P. Chopda, E. Wang, A. Bikumandla, A. Karthik Sengottuvel, K. Thottempudi, A. Narasimha, B. Dodds, C. Gao, J. Zhang, M. Al-Sanabani, A. Zehtabioskuie, J. Fix, H. Yu, R. Li, K. Gondkar, J. Montgomery, M. Tsai, S. Dwarakapuram, S. Desai, N. Avidan, P. Ramani, K. Narayanan, A. Mathews, S. Gopal, M. Naumov, V. Rao, K. Noru, H. Reddy, P. Venkatapuram, and A. Bjorlin, "Mtia: First generation silicon targeting meta's recommendation systems," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ser. ISCA '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3579371.3589348

[20] S. Gao, X. Chen, P. Li, Z. Ren, L. Bing, D. Zhao, and R. Yan, "Abstractive text summarization by incorporating reader comments," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 6399–6406.

[21] H. Genc, A. Haj-Ali, V. Iyer, A. Amid, H. Mao, J. C. Wright, C. Schmidt, J. Zhao, A. J. Ou, M. Banister, Y. S. Shao, B. Nikolic, I. Stoica, and K. Asanovic, "Gemmini: An agile systolic array generator enabling systematic evaluations of deep-learning architectures," *CoRR*, vol. abs/1911.09925, 2019. [Online]. Available: http://arxiv.org/abs/1911.09925

[22] K. Hegde, P.-A. Tsai, S. Huang, V. Chandra, A. Parashar, and C. W. Fletcher, "Mind mappings: enabling efficient algorithm-accelerator mapping space search," in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 943–958. [Online]. Available: https://doi.org/10.1145/3445814.3446762

[23] Q. Huang, M. Kang, G. Dinh, T. Norell, A. Kalaiah, J. Demmel, J. Wawrzynek, and Y. S. Shao, "Cosa: Scheduling by constrained optimization for spatial accelerators," in *Proceedings of the 48th Annual International Symposium on Computer Architecture*, ser. ISCA '21. IEEE Press, 2021, p. 554–566. [Online]. Available: https://doi.org/10.1109/ISCA52012.2021.00050

[24] G. Jeong, G. Kestor, P. Chatarasi, A. Parashar, P.-A. Tsai, S. Rajamanickam, R. Gioiosa, and T. Krishna, "Union: A unified hw-sw co-design ecosystem in mlir for evaluating tensor operations on spatial accelerators," in *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2021, pp. 30–44.

[25] H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and T. Zhao, "SMART: Robust and efficient fine-tuning for pre-trained natural language models through principled regularized optimization," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, Jul. 2020, pp. 2177–2190. [Online]. Available: https://aclanthology.org/2020.acl-main.197

[26] N. P. Jouppi, D. Hyun Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma, T. Norrie, N. Patil, S. Prasad, C. Young, Z. Zhou, and D. Patterson, "Ten lessons from three generations shaped google's tpuv4i : Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021, pp. 1–14.

[27] S.-C. Kao and T. Krishna, "Gamma: Automating the hw mapping of dnn models on accelerators via genetic algorithm," in *Proceedings of the 39th International Conference on Computer-Aided Design*, ser. ICCAD '20. New York, NY, USA: Association for Computing Machinery, 2020. [Online]. Available: https://doi.org/10.1145/3400302.3415639

[28] ——, "Magma: An optimization framework for mapping multiple dnns on multiple accelerator cores," in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022, pp. 814–830.

[29] S.-C. Kao, A. Parashar, P.-A. Tsai, and T. Krishna, "Demystifying map space exploration for npus," 2022.

[30] S.-C. Kao, M. Pellauer, A. Parashar, and T. Krishna, "Digamma: Domain-aware genetic algorithm for hw-mapping co-optimization for dnn accelerators," in *2022 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2022, pp. 232–237.

[31] D. Khudia, J. Huang, P. Basu, S. Deng, H. Liu, J. Park, and M. Smelyanskiy, "Fbgemm: Enabling high-performance low-precision deep learning inference," *arXiv preprint arXiv:2101.05615*, 2021.

[32] T. Krishna, H. Kwon, A. Parashar, M. Pellauer, and A. Samajdar, "Data orchestration in deep learning accelerators," 2020.

[33] H. Kwon, P. Chatarasi, V. Sarkar, T. Krishna, M. Pellauer, and A. Parashar, "Maestro: A data-centric approach to understand reuse, performance, and hardware cost of dnn mappings," *IEEE Micro*, vol. 40, no. 3, pp. 20–29, 2020.

[34] H. Kwon, M. Pellauer, A. Parashar, and T. Krishna, "Flexion: A quantitative metric for flexibility in dnn accelerators," *IEEE Computer Architecture Letters*, vol. 20, no. 1, pp. 1–4, 2021.

[35] H. Kwon, A. Samajdar, and T. Krishna, "MAERI: Enabling Flexible Dataflow Mapping over DNN Accelerators via Reconfigurable Interconnects," in *Proceedings of the 23rd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2018.

[36] C. E. Leiserson, "Fat-trees: universal networks for hardware-efficient supercomputing," *IEEE transactions on Computers*, vol. 100, no. 10, pp. 892–901, 1985.
[37] W. Liu, S. Liao, W. Ren, W. Hu, and Y. Yu, "High-level semantic feature detection: A new perspective for pedestrian detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 5187–5196.
[38] D. Matani, "Efficient pytorch: Tensor memory format matters." [Online]. Available: https://pytorch.org/blog/tensor-memory-format-matters/
[39] NVIDIA. (2016) NVIDIA Deep Learning Accelerator (NVDLA). [Online]. Available: http://nvdla.org/primer.html
[40] K. Ovtcharov, O. Ruwase, J.-Y. Kim, J. Fowers, K. Strauss, and E. S. Chung, "Accelerating deep convolutional neural networks using specialized hardware," *Microsoft Research Whitepaper*, vol. 2, no. 11, pp. 1–4, 2015.
[41] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A Systematic Approach to DNN Accelerator Evaluation," in *Proceedings of the International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019.
[42] E. Qin, A. Samajdar, H. Kwon, V. Nadella, S. Srinivasan, D. Das, B. Kaul, and T. Krishna, "Sigma: A sparse and irregular gemm accelerator with flexible interconnects for dnn training," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2020, pp. 58–70.
[43] A. Reuther, P. Michaleas, M. Jones, V. Gadepally, S. Samsi, and J. Kepner, "Ai and ml accelerator survey and trends," 2022. [Online]. Available: https://arxiv.org/abs/2210.04055
[44] A. Samajdar, M. Pellauer, and T. Krishna, "Self-adaptive reconfigurable arrays (sara): Using ml to assist scaling gemm acceleration," *ArXiv*, vol. abs/2101.04799, 2021.
[45] A. Samajdar, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "SCALE-Sim: Systolic CNN Accelerator Simulator," *arXiv preprint arXiv:1811.02883*, 2018.
[46] K. Seshadri, B. Akin, J. Laudon, R. Narayanaswami, and A. Yazdanbakhsh, "An evaluation of edge tpu accelerators for convolutional neural networks," 2022.
[47] Y. S. Shao, J. Clemons, R. Venkatesan, B. Zimmer, M. Fojtik, N. Jiang, B. Keller, A. Klinefelter, N. Pinckney, P. Raina, S. G. Tell, Y. Zhang, W. J. Dally, J. Emer, C. T. Gray, B. Khailany, and S. W. Keckler, "Simba: Scaling deep-learning inference with multi-chip-module-based architecture," in *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '52. New York, NY, USA: Association for Computing Machinery, 2019, p. 14–27. [Online]. Available: https://doi.org/10.1145/3352460.3358302
[48] Y. Shen, T. Ji, M. Ferdman, and P. Milder, "Medusa: A scalable interconnect for many-port dnn accelerators and wide dram controller interfaces," in *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*, 2018, pp. 101–1014.
[49] H. Tao, M. M. Hameed, H. A. Marhoon, M. Zounemat-Kermani, S. Heddam, S. Kim, S. O. Sulaiman, M. L. Tan, Z. Sa'adi, A. D. Mehr, M. F. Allawi, S. Abba, J. M. Zain, M. W. Falah, M. Jamei, N. D. Bokde, M. Bayatvarkeshi, M. Al-Mukhtar, S. K. Bhagat, T. Tiyasha, K. M. Khedher, N. Al-Ansari, S. Shahid, and Z. M. Yaseen, "Groundwater level prediction using machine learning models: A comprehensive review," *Neurocomputing*, vol. 489, pp. 271–308, 2022. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S092523122200282X
[50] J. Weng, S. Liu, V. Dadu, Z. Wang, P. Shah, and T. Nowatzki, "Dsagen: Synthesizing programmable spatial accelerators," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, 2020, pp. 268–281.
[51] Xilinx. (2022) Xilinx Deep Learning Unit (DPU). [Online]. Available: https://docs.xilinx.com/r/en-US/ug1414-vitis-ai/Deep-Learning-Processor-Unit